



JavaScript: Functions



Outline

- ▶ Construct programs modularly from small pieces called functions.
- ▶ Define new functions.
- ▶ Pass information between functions.
- ▶ Use simulation techniques based on random number generation.
- ▶ Use the HTML audio and video elements.
- ▶ Use additional global methods.



9.1 Introduction

- ▶ To develop and maintain a large program
 - construct it from small, simple pieces
 - divide and conquer



Function v.s. Method

Function:

`total += parseInt(inputValue, 10)`

Method:

`window.alert("Hi")`



9.2 Functions in JavaScript

- ▶ You can define **programmer-defined functions** that perform specific tasks and use them at many points in a script
 - The actual statements defining the function are written only once and are hidden from other functions
- ▶ Functions are invoked by writing the name of the function, followed by a left parenthesis, followed by a comma-separated list of zero or more arguments, followed by a right parenthesis
- ▶ Methods are called in the same way as functions, but require the name of the object to which the method belongs and a dot preceding the method name
- ▶ Function (and method) arguments may be **constants, variables or expressions**



9.3.1 Programmer–Defined Function square

- ▶ return statement
 - passes information from inside a function back to the point in the program where it was called
- ▶ A function must be called explicitly for the code in its body to execute
- ▶ The format of a function definition is

*function function-name(parameter-list) {
 declarations and statements
}*



```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.2: SquareInt.html -->
4  <!-- Programmer-defined function square. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>A Programmer-Defined square Function</title>
9          <style type = "text/css">
10             p { margin: 0; }
11          </style>
12          <script>
13
14             document.writeln( "<h1>Square the numbers from 1 to 10</h1>" );
15
16             // square the numbers from 1 to 10
17             for ( var x = 1; x <= 10; ++x )
18                 document.writeln( "<p>The square of " + x + " is " +
19                     square( x ) + "</p>" );
20
```

Fig. 9.2 | Programmer-defined function square. (Part I of 3.)



```
21      // The following square function definition's body is executed
22      // only when the function is called explicitly as in line 19
23      function square( y )
24      {
25          return y * y;
26      } // end function square
27
28      </script>
29      </head><body></body> <!-- empty body element -->
30  </html>
```

Fig. 9.2 | Programmer-defined function square. (Part 2 of 3.)

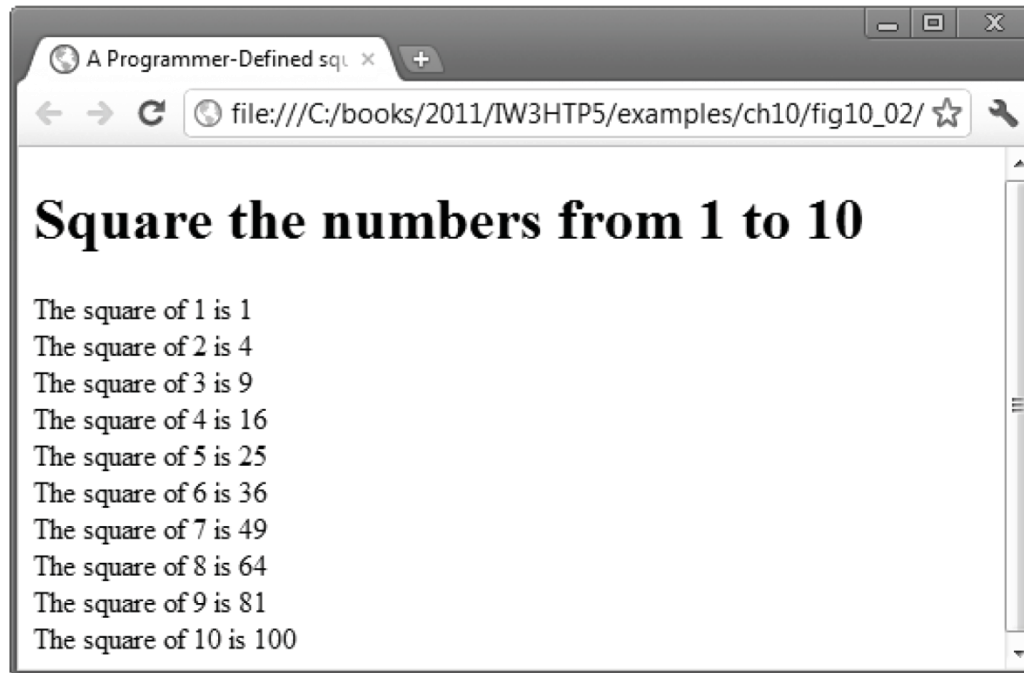


Fig. 9.2 | Programmer-defined function square. (Part 3 of 3.)



9.3.1 Programmer–Defined Function square (cont.)

- ▶ Three ways to return control to the point at which a function was invoked
 - Reaching the function–ending right brace
 - Executing the statement `return;`
 - Executing the statement `return expression;` to return the value of *expression* to the caller
- ▶ When a return statement executes, control returns immediately to the point at which the function was invoked

9.4 Notes on Programmer–Defined Functions

- ▶ All variables declared with the keyword `var` in function definitions are **local variables** (包含迴圈內的變數)
 - they can be accessed only in the function in which they're defined.
- ▶ A function's parameters are also considered to be local variables.
- ▶ There are several reasons for modularizing a program with functions.
 - Divide-and-conquer approach makes program development more manageable.
 - Software reusability.
 - Avoid repeating code in a program.



Common Programming Error

Using the JavaScript `var` or `let` keyword to declare a variable in a function parameter list results in a JavaScript runtime error.

(要花一點時間搞清楚JavaScript與C/Java 之語法差異)



Software Engineering Observation

To promote software reusability, every function should be limited to **performing a single, well-defined task, and the name of the function should express that task effectively**. Such functions make programs easier to write, debug, maintain and modify.

小而功能明確的function易於test與debug

9.5 Random Number Generation

- ▶ random method generates a **floating-point value from 0.0 up to, but *not* including, 1.0**
- ▶ Random integers in a certain range can be generated by scaling and shifting the values returned by random, then using `Math.floor` to convert them to integers
 - The **scaling factor determines the size** of the range (i.e. a scaling factor of 4 means four possible integers)
 - The **shift number is added to the result to determine where the range begins** (i.e. shifting the numbers by 3 would give numbers between 3 and 7)
- ▶ Method `Math.floor` rounds its argument down to the closest integer

```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.4: RandomInt.html -->
4  <!-- Random integers, shifting and scaling. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>Shifted and Scaled Random Integers</title>
9          <style type = "text/css">
10             p, ol { margin: 0; }
11             li    { display: inline; margin-right: 10px; }
12          </style>
13          <script>
14
15             var value;
16
17             document.writeln( "<p>Random Numbers</p><ol>" );
18
19             for ( var i = 1; i <= 30; ++i )
20             {
21                 value = Math.floor( 1 + Math.random() * 6 );
22                 document.writeln( "<li>" + value + "</li>" );
23             } // end for
24
```

Fig. 9.4 | Random integers, shifting and scaling. (Part I of 2.)

```
25         document.writeln( "</ol>" );
26
27     </script>
28 </head><body></body>
29 </html>
```

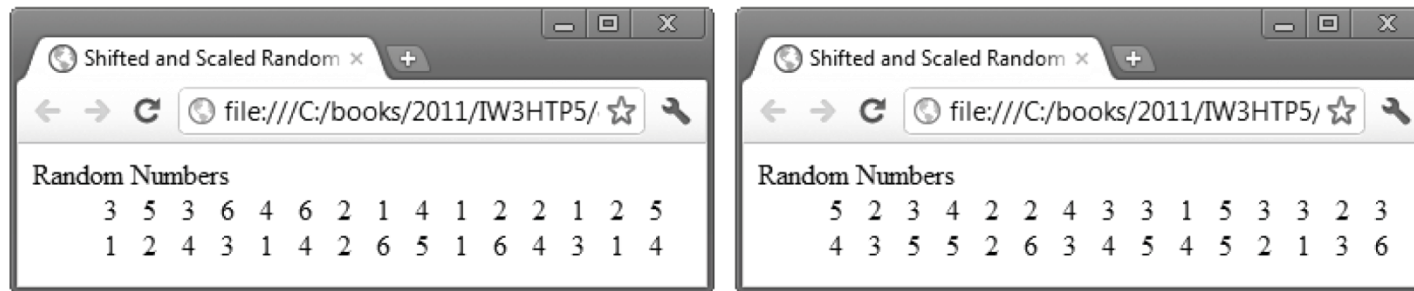


Fig. 9.4 | Random integers, shifting and scaling. (Part 2 of 2.)

9.5.2 Displaying Random Images

- ▶ In the next example, we build a **random image generator**—a script that displays four randomly selected die images every time the user clicks a Roll Dice button on the page.
- ▶ For the script in Fig. 9.5 to function properly, the directory containing the file `RollDice.html` must also contain the six die images—these are included with this chapter's examples.

```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.5: RollDice.html -->
4  <!-- Random dice image generation using Math.random. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>Random Dice Images</title>
9          <style type = "text/css">
10             li { display: inline; margin-right: 10px; }
11             ul { margin: 0; }
12          </style>
13          <script>
14             // variables used to interact with the i mg elements
15             var die1Image;
16             var die2Image;
17             var die3Image;
18             var die4Image;
19
```

Fig. 9.5 | Random dice image generation using `Math.random`. (Part 1 of 4.)

```
20 // register button listener and get the img elements
21 function start()
22 {
23     var button = document.getElementById( "rollButton" );
24     button.addEventListener( "click", rollDice, false );
25     die1Image = document.getElementById( "die1" );
26     die2Image = document.getElementById( "die2" );
27     die3Image = document.getElementById( "die3" );
28     die4Image = document.getElementById( "die4" );
29 } // end function rollDice
30
31 // roll the dice
32 function rollDice()
33 {
34     setImage( die1Image );
35     setImage( die2Image );
36     setImage( die3Image );
37     setImage( die4Image );
38 } // end function rollDice
39
```

Fig. 9.5 | Random dice image generation using `Math.random`. (Part 2 of 4.)



```
40      // set image source for a die
41      function setImage( dieImg )
42      {
43          var dieValue = Math.floor( 1 + Math.random() * 6 );
44          dieImg.setAttribute( "src", "die" + dieValue + ".png" );
45          dieImg.setAttribute( "alt",
46              "die image with " + dieValue + " spot(s)" );
47      } // end function setImage
48
49      window.addEventListener( "load", start, false );
50  </script>
51  </head>
```

Fig. 9.5 | Random dice image generation using `Math.random`. (Part 3 of 4.)


```

52 <body>
53   <form action = "#">
54     <input id = "rollButton" type = "button" value = "Roll Dice">
55   </form>
56   <ol>
57     <li><img id = "die1" src = "blank.png" alt = "die 1 image"></li>
58     <li><img id = "die2" src = "blank.png" alt = "die 2 image"></li>
59     <li><img id = "die3" src = "blank.png" alt = "die 3 image"></li>
60     <li><img id = "die4" src = "blank.png" alt = "die 4 image"></li>
61   </ol>
62 </body>
63 </html>

```



Fig. 9.5 | Random dice image generation using `Math.random`. (Part 4 of 4.)

9.5.2 Displaying Random Images

User Interactions Via Event Handling

- ▶ Until now, all user interactions with scripts have been through a **prompt dialog** or an **alert dialog**.
 - ▶ These dialogs are valid ways to receive input from a user and to display messages, but they're fairly limited in their capabilities.
 - ▶ A prompt dialog can obtain only one value at a time from the user, and a message dialog can display only one message.

9.5.2 Displaying Random Images

- ▶ This program uses an HTML form and a new graphical user interface concept— **GUI event handling**.
- ▶ Inputs are typically received from the user **via an HTML form**.
 - ▶ 事件通常源自使用者的操作，但也可能來自系統行為，像是load事件就是網頁載入完畢後驅動。
- ▶ Outputs are typically displayed to the user **in the web page**.
- ▶ The JavaScript executes in response to the user's interaction with an element in a form.
 - ▶ **This interaction causes an event.**
- ▶ Scripts are often used to respond to user initiated events.
- ▶ 事件(event)就像一個信號，例如下課結束會有鐘聲(event)，接著就會開始上課(指定要執行的function)。

9.5.2 Displaying Random Images

The body Element

- ▶ The elements in the body are used extensively in the script.

The form Element

- ▶ A form may contain an action attribute
 - ▶ Because this form does not post its information to a web server, the string “#” is used simply to allow this document to validate.
 - ▶ The # symbol by itself represents the current page.
 - ▶ 一般action attribute會指向一個server script的URL
- ▶ 若以 JavaScript 處理表單提交，action 可留空或省略，並在 submit handler中呼叫e.preventDefault()，阻止瀏覽器對事件的預設行為。

9.5.2 Displaying Random Images

The button input Element and Event-Driven Programming

- ▶ **Event-driven programming**
 - ▶ the user interacts with an element in the web page, the script is notified of the event and the script processes the event.
- ▶ The user's interaction with the GUI “**drives**” the program.
- ▶ The button click is known as the **event**.
- ▶ The function that's called when an event occurs is known as an **event handler**.



9.5.2 Displaying Random Images

- ▶ When a GUI event occurs in a form, the browser calls the specified event-handling function.
- ▶ Before any event can be processed, each element must know which event-handling function will be called when a particular event occurs.

9.5.2 Displaying Random Images

The img Elements

- ▶ The four `img` elements will display the four randomly selected dice.
- ▶ Their `id` attributes (`die1`, `die2`, `die3` and `die4`, respectively) can be used to apply CSS styles and to enable script code to refer to these element in the HTML document.
- ▶ Because the `id` attribute, if specified, must have a unique value among all `id` attributes in the page, Java-Script can reliably refer to any single element via its `id` attribute.
- ▶ Each `img` element displays the image `blank.png` (an empty white image) when the page first renders.

9.5.2 Displaying Random Images

Specifying a Function to Call When the Browser Finishes Loading a Document

- ▶ Many examples will execute a JavaScript function when the document finishes loading.
- ▶ This can be accomplished by handling the window object's load event.
- ▶ To specify the function to call when an event occurs, you register an event handler for that event.

Two options:

- ▶ *`window.addEventListener('load', start);`*
- ▶ *`document.addEventListener('DOMContentLoaded', start);`*

9.5.2 Displaying Random Images

Method `addEventListener` is available for every DOM node. The method takes three arguments:

- ▶ The first is the name of the event for which we're registering a handler
- ▶ The second is the function that will be called to handle the event
- ▶ The last argument is typically false (目前都設定為false即可)
 - ▶ true: event capture model
 - ▶ false: event bubbling model

9.5.3 Rolling Dice Repeatedly and Displaying Statistics



- ▶ To show that the random values representing the dice occur with approximately equal likelihood, let's allow the user to roll 12 dice at a time and keep statistics showing the number of times each face occurs and the percentage of the time each face is rolled (Fig. 9.6).

存取欄位或區塊的內容

- ▶ The **getElementById** method, given an **id** as an argument, **finds the HTML element** with a matching **id** attribute and **returns a JavaScript object** representing the element.
- ▶ The **value** property of a JavaScript object representing **an HTML text input element** specifies the text to display in the text field.
 - ▶ 取得與設定文字欄位內容，透過value屬性(值為純文字)
- ▶ An HTML container (e.g. **div**, **span**, **p**) element's **innerHTML** property can be used in a script to set the contents of the element.
 - ▶ 取得與設定div內容，透過innerHTML屬性(值為HTML片段)

```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.6: RollDice.html -->
4  <!-- Rolling 12 dice and displaying frequencies. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>Die Rolling Frequencies</title>
9          <style type = "text/css">
10             img          { margin-right: 10px; }
11             table         { width: 200px;
12                             border-collapse: collapse;
13                             background-color: lightblue; }
14             table, td, th { border: 1px solid black;
15                             padding: 4px;
16                             margin-top: 20px; }
17             th            { text-align: left;
18                             color: white;
19                             background-color: darkblue; }
20          </style>
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part I of 9.)

```
21      <script>
22          var frequency1 = 0;
23          var frequency2 = 0;
24          var frequency3 = 0;
25          var frequency4 = 0;
26          var frequency5 = 0;
27          var frequency6 = 0;
28          var totalDice = 0;
29
30          // register button event handler
31          function start()
32          {
33              var button = document.getElementById( "rollButton" );
34              button.addEventListener( "click", rollDice, false );
35          } // end function start
36
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 2 of 9.)



```
37 // roll the dice
38 function rollDice()
39 {
40     var face; // face rolled
41
42     // loop to roll die 12 times
43     for ( var i = 1; i <= 12; ++i )
44     {
45         face = Math.floor( 1 + Math.random() * 6 );
46         tallyRolls( face ); // increment a frequency counter
47         setImage( i, face ); // display appropriate die image
48         ++totalDice; // increment total
49     } // end die rolling loop
50
51     updateFrequencyTable();
52 } // end function rollDice
53
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 3 of 9.)

```
54 // increment appropriate frequency counter
55 function tallyRolls( face )
56 {
57     switch ( face )
58     {
59         case 1:
60             ++frequency1;
61             break;
62         case 2:
63             ++frequency2;
64             break;
65         case 3:
66             ++frequency3;
67             break;
68         case 4:
69             ++frequency4;
70             break;
71         case 5:
72             ++frequency5;
73             break;
74         case 6:
75             ++frequency6;
76             break;
77     } // end switch
78 } // end function tallyRolls
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 4 of 9.)



```
79
80 // set image source for a die
81 function setImage( dieNumber, face )
82 {
83     var dieImg = document.getElementById( "die" + dieNumber );
84     dieImg.setAttribute( "src", "die" + face + ".png" );
85     dieImg.setAttribute( "alt", "die with " + face + " spot(s)" );
86 } // end function setImage
87
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 5 of 9.)



```
88 // update frequency table in the page
89 function updateFrequencyTable()
90 {
91     var tableDiv = document.getElementById( "frequencyTableDiv" );
92
93     tableDiv.innerHTML = "<table>" +
94         "<caption>Die Rolling Frequencies</caption>" +
95         "<thead><th>Face</th><th>Frequency</th>" +
96         "<th>Percent</th></thead>" +
97         "<tbody><tr><td>1</td><td>" + frequency1 + "</td><td>" +
98         formatPercent(frequency1 / totalDice) + "</td></tr>" +
99         "<tr><td>2</td><td>" + frequency2 + "</td><td>" +
100         formatPercent(frequency2 / totalDice) + "</td></tr>" +
101         "<tr><td>3</td><td>" + frequency3 + "</td><td>" +
102         formatPercent(frequency3 / totalDice) + "</td></tr>" +
103         "<tr><td>4</td><td>" + frequency4 + "</td><td>" +
104         formatPercent(frequency4 / totalDice) + "</td></tr>" +
105         "<tr><td>5</td><td>" + frequency5 + "</td><td>" +
106         formatPercent(frequency5 / totalDice) + "</td></tr>" +
107         "<tr><td>6</td><td>" + frequency6 + "</td><td>" +
108         formatPercent(frequency6 / totalDice) + "</td></tr>" +
109         "</tbody></table>";
110 } // end function updateFrequencyTable
111
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 6 of 9.)



```
112      // format percentage
113      function formatPercent( value )
114      {
115          value *= 100;
116          return value.toFixed(2);
117      } // end function formatPercent
118
119      window.addEventListener( "load", start, false );
120  </script>
121  </head>
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 7 of 9.)



```
I22 <body>
I23   <p><img id = "die1" src = "blank.png" alt = "die 1 image">
I24     <img id = "die2" src = "blank.png" alt = "die 2 image">
I25     <img id = "die3" src = "blank.png" alt = "die 3 image">
I26     <img id = "die4" src = "blank.png" alt = "die 4 image">
I27     <img id = "die5" src = "blank.png" alt = "die 5 image">
I28     <img id = "die6" src = "blank.png" alt = "die 6 image"></p>
I29   <p><img id = "die7" src = "blank.png" alt = "die 7 image">
I30     <img id = "die8" src = "blank.png" alt = "die 8 image">
I31     <img id = "die9" src = "blank.png" alt = "die 9 image">
I32     <img id = "die10" src = "blank.png" alt = "die 10 image">
I33     <img id = "die11" src = "blank.png" alt = "die 11 image">
I34     <img id = "die12" src = "blank.png" alt = "die 12 image"></p>
I35   <form action = "#">
I36     <input id = "rollButton" type = "button" value = "Roll Dice">
I37   </form>
I38   <div id = "frequencyTableDiv"></div>
I39 </body>
I40 </html>
```

Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 8 of 9.)

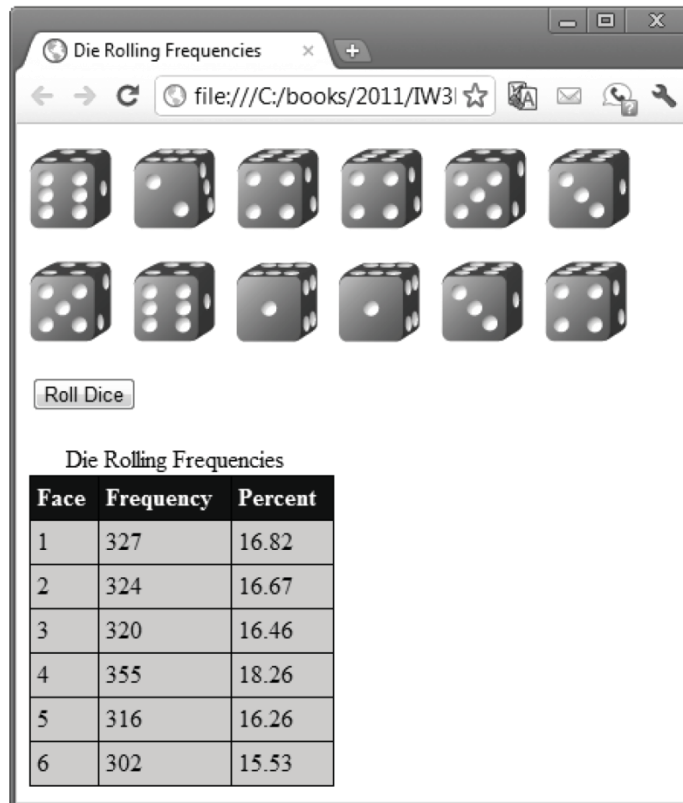


Fig. 9.6 | Rolling 12 dice and displaying frequencies. (Part 9 of 9.)

9.5.3 Rolling Dice Repeatedly and Displaying Statistics



- ▶ We can generalize this result as
$$\text{face} = \text{Math.floor}(a + \text{Math.random()} * b);$$
- ▶ where a is the shifting value (which is equal to the first number in the desired range of consecutive integers) and b is the scaling factor (which is equal to the width of the desired range of consecutive integers).



9.7 Scope Rules

- ▶ Each identifier in a program has a scope
- ▶ The **scope** of an identifier for a variable or function is the portion of the program in which the identifier can be referenced
- ▶ **Global variables** or **script-level variables** are accessible in any part of a script and are said to have **global scope**
 - Thus every function in the script can potentially use the variables

9.7 Scope Rules (Cont.)

- ▶ Identifiers declared inside a function have **function (or local) scope** and can be used only in that function (包含迴圈內的變數) (此為var所宣告之變數)
- ▶ Function scope begins with the opening left brace ({} of the function in which the identifier is declared and ends at the terminating right brace (})
- ▶ Local variables of a function and function parameters have function scope
- ▶ If a local variable in a function has the same name as a global variable, the global variable is “hidden” from the body of the function.



Good Programming Practice

Avoid local-variable names that hide global-variable names. This can be accomplished by simply avoiding the use of duplicate identifiers in a script.



```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.9: scoping.html -->
4  <!-- Scoping example. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>Scoping Example</title>
9          <style type = "text/css">
10              p      { margin: 0px; }
11              p.space { margin-top: 10px; }
12          </style>
```

Fig. 9.9 | Scoping example. (Part I of 4.)



```
13 <script>
14     var output; // stores the string to display
15     var x = 1; // global variable
16
17     function start()
18     {
19         var x = 5; // variable local to function start
20
21         output = "<p>local x in start is " + x + "</p>";
22
23         functionA(); // functionA has local x
24         functionB(); // functionB uses global variable x
25         functionA(); // functionA reinitializes local x
26         functionB(); // global variable x retains its value
27
28         output += "<p class='space'>local x in start is " + x +
29                 "</p>";
30         document.getElementById( "results" ).innerHTML = output;
31     } // end function start
32
```

Fig. 9.9 | Scoping example. (Part 2 of 4.)



```
33  function functionA()  
34  {  
35      var x = 25; // initialized each time functionA is called  
36  
37      output += "<p class='space'>local x in functionA is " + x +  
38          " after entering functionA</p>";  
39      ++x;  
40      output += "<p>local x in functionA is " + x +  
41          " before exiting functionA</p>";  
42  } // end functionA  
43  
44  function functionB()  
45  {  
46      output += "<p class='space'>global variable x is " + x +  
47          " on entering functionB";  
48      x *= 10;  
49      output += "<p>global variable x is " + x +  
50          " on exiting functionB</p>";  
51  } // end functionB  
52  
53  window.addEventListener("load", start, false );  
54  </script>  
55  </head>
```

Fig. 9.9 | Scoping example. (Part 3 of 4.)

```
56     <body>
57         <div id = "results"></div>
58     </body>
59 </html>
```

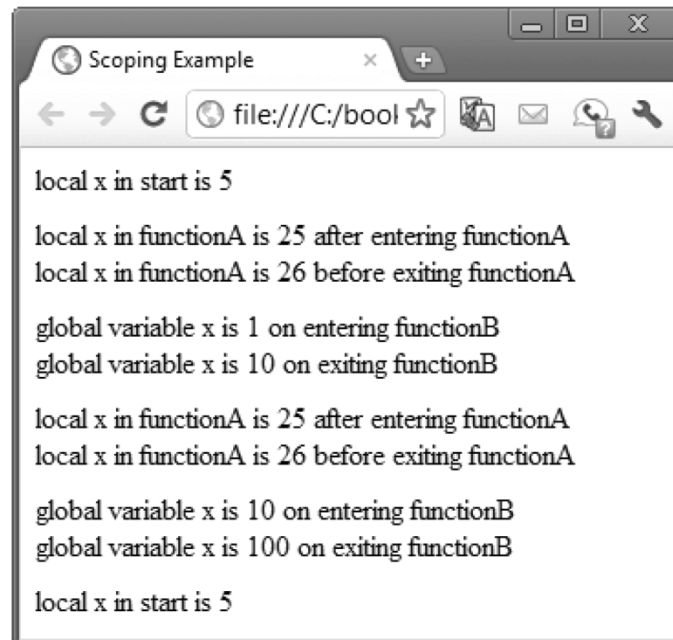


Fig. 9.9 | Scoping example. (Part 4 of 4.)

9.8 JavaScript Global Functions

- ▶ JavaScript provides global functions as part of a `Global` object
- ▶ This object contains
 - all the global variables in the script
 - all the user-defined functions in the script
 - all the built-in global functions listed in the following slide
- ▶ You do not need to use the `Global` object directly; JavaScript uses it for you



Global function	Description
<code>isFinite</code>	Takes a numeric argument and returns <code>true</code> if the value of the argument is not <code>NaN</code> , <code>Number.POSITIVE_INFINITY</code> or <code>Number.NEGATIVE_INFINITY</code> (values that are not numbers or numbers outside the range that JavaScript supports)—otherwise, the function returns <code>false</code> .
<code>isNaN</code>	Takes a numeric argument and returns <code>true</code> if the value of the argument is not a number; otherwise, it returns <code>false</code> . The function is commonly used with the return value of <code>parseInt</code> or <code>parseFloat</code> to determine whether the result is a proper numeric value.
<code>parseFloat</code>	Takes a string argument and attempts to convert the <i>beginning</i> of the string into a floating-point value. If the conversion is unsuccessful, the function returns <code>NaN</code> ; otherwise, it returns the converted value (e.g., <code>parseFloat("abc123.45")</code> returns <code>NaN</code> , and <code>parseFloat("123.45abc")</code> returns the value <code>123.45</code>).

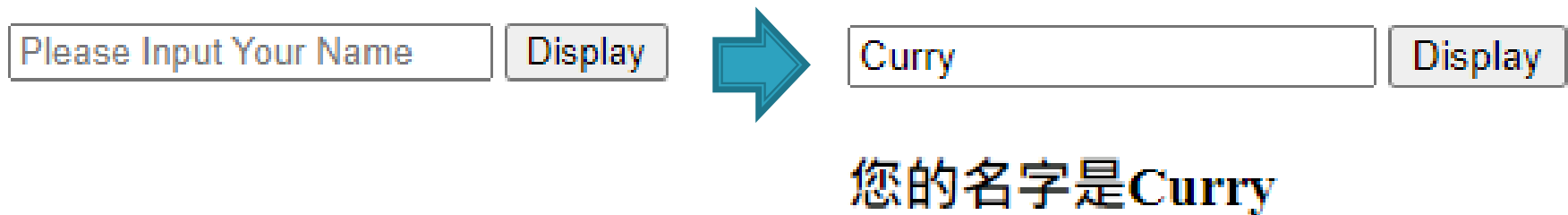
Fig. 9.10 | JavaScript global functions. (Part I of 2.)



Global function	Description
<code>parseInt</code>	Takes a string argument and attempts to convert the beginning of the string into an integer value. If the conversion is unsuccessful, the function returns NaN; otherwise, it returns the converted value (for example, <code>parseInt("abc123")</code> returns NaN, and <code>parseInt("123abc")</code> returns the integer value 123). This function takes an optional second argument, from 2 to 36, specifying the radix (or base) of the number. Base 2 indicates that the first argument string is in binary format, base 8 that it's in octal format and base 16 that it's in hexadecimal format. See Appendix E, for more information on binary, octal and hexadecimal numbers.

Fig. 9.10 | JavaScript global functions. (Part 2 of 2.)

如何以JavaScript取得輸入元素的值



// 設計按鈕事件處理器的function

```
function start() {  
    //找到id為"displayButton"的網頁元素物件 (就是那個按鈕的物件)  
    const button = document.getElementById("displayButton");  
    //幫按鈕註冊事件處理器，一旦按鈕被使用者點擊，就會呼叫display function  
    button.addEventListener("click", display, false);  
}
```




如何以JavaScript取得輸入元素的值₂

// 顯示訊息的function

```
function display() {  
    //找到id為"name"的網頁元素物件 (就是那個文字欄位的物件)  
    let textFieldObject = document.getElementById("name");  
    //取得文字欄位的文字內容  
    let content = textFieldObject.value;  
  
    //組合出想在div區塊中呈現的html片段  
    let message = "<h3>您的名字是" + content + "</h3>";  
  
    //找到id為"display"的網頁元素物件 (就是那個div區塊的物件)  
    const divObject = document.getElementById("display");  
  
    //將div區塊的內容改為我們準備好的html片段，網頁上的div就會顯示新的內容  
    divObject.innerHTML = message;  
}
```



9.6 Example: Game of Chance; Introducing the HTML audio and video Elements

```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.7: Craps.html -->
4  <!-- Craps game simulation. -->
5  <html>
6    <head>
7      <meta charset = "utf-8">
8      <title>Craps Game Simulation</title>
9      <style type = "text/css">
10         p.red { color: red }
11         img   { width: 54px; height: 54px; }
12         div   { border: 5px ridge royalblue;
13                padding: 10px; width: 120px;
14                margin-bottom: 10px; }
15         .point { margin: 0px; }
16      </style>
```

Fig. 9.7 | Craps game simulation. (Part I of 12.)

```
17     <script>
18         // variables used to refer to page elements
19         var pointDie1Img; // refers to first die point img
20         var pointDie2Img; // refers to second die point img
21         var rollDie1Img; // refers to first die roll img
22         var rollDie2Img; // refers to second die roll img
23         var messages; // refers to "messages" paragraph
24         var playButton; // refers to Play button
25         var rollButton; // refers to Roll button
26         var dicerolling; // refers to audio clip for dice
27
28         // other variables used in program
29         var myPoint; // point if no win/loss on first roll
30         var die1Value; // value of first die in current roll
31         var die2Value; // value of second die in current roll
32
```

Fig. 9.7 | Craps game simulation. (Part 2 of 12.)

```
33 // starts a new game
34 function startGame()
35 {
36     // get the page elements that we'll interact with
37     dicerolling = document.getElementById( "dicerolling" );
38     pointDie1Img = document.getElementById( "pointDie1" );
39     pointDie2Img = document.getElementById( "pointDie2" );
40     rollDie1Img = document.getElementById( "rollDie1" );
41     rollDie2Img = document.getElementById( "rollDie2" );
42     messages = document.getElementById( "messages" );
43     playButton = document.getElementById( "play" );
44     rollButton = document.getElementById( "roll" );
45
46     // prepare the GUI
47     rollButton.disabled = true; // disable rollButton
48     setImage( pointDie1Img ); // reset image for new game
49     setImage( pointDie2Img ); // reset image for new game
50     setImage( rollDie1Img ); // reset image for new game
51     setImage( rollDie2Img ); // reset image for new game
52
53     myPoint = 0; // there is currently no point
54     firstRoll(); // roll the dice to start the game
55 } // end function startGame
56
```

Fig. 9.7 | Craps game simulation. (Part 3 of 12.)



```
57 // perform first roll of the game
58 function firstRoll()
59 {
60     var sumOfDice = rollDice(); // first roll of the dice
61
62     // determine if the user won, lost or must continue rolling
63     switch (sumOfDice)
64     {
65         case 7: case 11: // win on first roll
66             messages.innerHTML =
67                 "You Win!!! Click Play to play again.";
68             break;
69         case 2: case 3: case 12: // lose on first roll
70             messages.innerHTML =
71                 "Sorry. You Lose. Click Play to play again.";
72             break;
73         default: // remember point
74             myPoint = sumOfDice;
75             setImage( pointDie1Img, die1Value );
76             setImage( pointDie2Img, die2Value );
77             messages.innerHTML = "Roll Again!";
78             rollButton.disabled = false; // enable rollButton
79             playButton.disabled = true; // disable playButton
80             break;
81     } // end switch
82 } // end function firstRoll
```

Fig. 9.7 | Craps game simulation. (Part 4 of 12.)

```
83
84 // called for subsequent rolls of the dice
85 function rollAgain()
86 {
87     var sumOfDice = rollDice(); // subsequent roll of the dice
88
89     if (sumOfDice == myPoint)
90     {
91         messages.innerHTML =
92             "You Win!!! Click Play to play again.";
93         rollButton.disabled = true; // disable rollButton
94         playButton.disabled = false; // enable playButton
95     } // end if
96     else if (sumOfDice == 7) // craps
97     {
98         messages.innerHTML =
99             "Sorry. You Lose. Click Play to play again.";
100         rollButton.disabled = true; // disable rollButton
101         playButton.disabled = false; // enable playButton
102     } // end else if
103 } // end function rollAgain
104
```

Fig. 9.7 | Craps game simulation. (Part 5 of 12.)



```
105 // roll the dice
106 function rollDice()
107 {
108     dicerolling.play(); // play dice rolling sound
109
110     // clear old die images while rolling sound plays
111     die1Value = NaN;
112     die2Value = NaN;
113     showDice();
114
115     die1Value = Math.floor(1 + Math.random() * 6);
116     die2Value = Math.floor(1 + Math.random() * 6);
117     return die1Value + die2Value;
118 } // end function rollDice
119
```

Fig. 9.7 | Craps game simulation. (Part 6 of 12.)



```
I20      // display rolled dice
I21      function showDice()
I22      {
I23          setImage( rollDie1Img, die1Value );
I24          setImage( rollDie2Img, die2Value );
I25      } // end function showDice
I26
I27      // set image source for a die
I28      function setImage( dieImg, dieValue )
I29      {
I30          if ( isFinite( dieValue ) )
I31              dieImg.src = "die" + dieValue + ".png";
I32          else
I33              dieImg.src = "blank.png";
I34      } // end function setImage
I35
```

Fig. 9.7 | Craps game simulation. (Part 7 of 12.)



```
I36      // register event listeners
I37      function start()
I38      {
I39          var playButton = document.getElementById( "play" );
I40          playButton.addEventListener( "click", startGame, false );
I41          var rollButton = document.getElementById( "roll" );
I42          rollButton.addEventListener( "click", rollAgain, false );
I43          var diceSound = document.getElementById( "dicerolling" );
I44          diceSound.addEventListener( "ended", showDice, false );
I45      } // end function start
I46
I47      window.addEventListener( "load", start, false );
I48      </script>
I49  </head>
I50  <body>
I51      <audio id = "dicerolling" preload = "auto">
I52          <source src = "http://test.deitel.com/dicerolling.mp3"
I53              type = "audio/mpeg">
I54          <source src = "http://test.deitel.com/dicerolling.ogg"
I55              type = "audio/ogg">
I56      Browser does not support audio tag</audio>
```

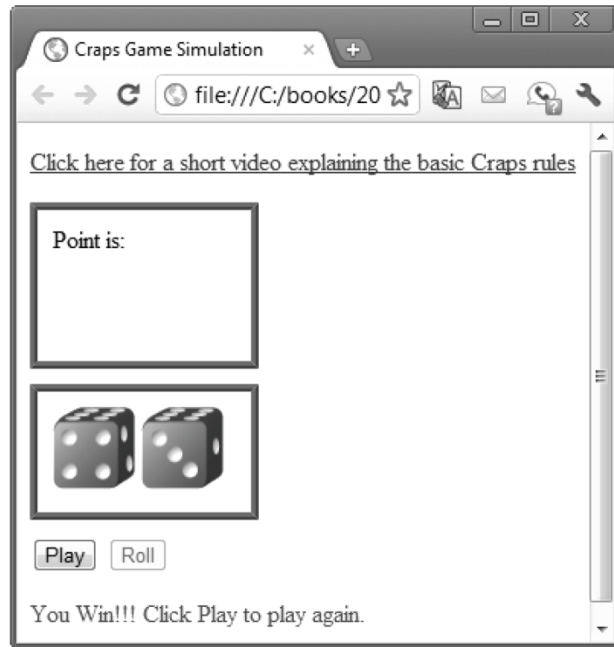
Fig. 9.7 | Craps game simulation. (Part 8 of 12.)



```
157 <p><a href = "CrapsRules.html">Click here for a short video
158     explaining the basic Craps rules</a></p>
159 <div id = "pointDiv">
160     <p class = "point">Point is:</p>
161     <img id = "pointDie1" src = "blank.png"
162         alt = "Die 1 of Point Value">
163     <img id = "pointDie2" src = "blank.png"
164         alt = "Die 2 of Point Value">
165 </div>
166 <div class = "rollDiv">
167     <img id = "rollDie1" src = "blank.png"
168         alt = "Die 1 of Roll Value">
169     <img id = "rollDie2" src = "blank.png"
170         alt = "Die 2 of Roll Value">
171 </div>
172 <form action = "#">
173     <input id = "play" type = "button" value = "Play">
174     <input id = "roll" type = "button" value = "Roll">
175 </form>
176 <p id = "messages" class = "red">Click Play to start the game</p>
177 </body>
178 </html>
```

Fig. 9.7 | Craps game simulation. (Part 9 of 12.)

a) Win on the first roll. In this case, the `pointDiv` does not show any dice and the **Roll** button



b) Loss on the first roll. In this case, the `pointDiv` does not show any dice and the **Roll** button

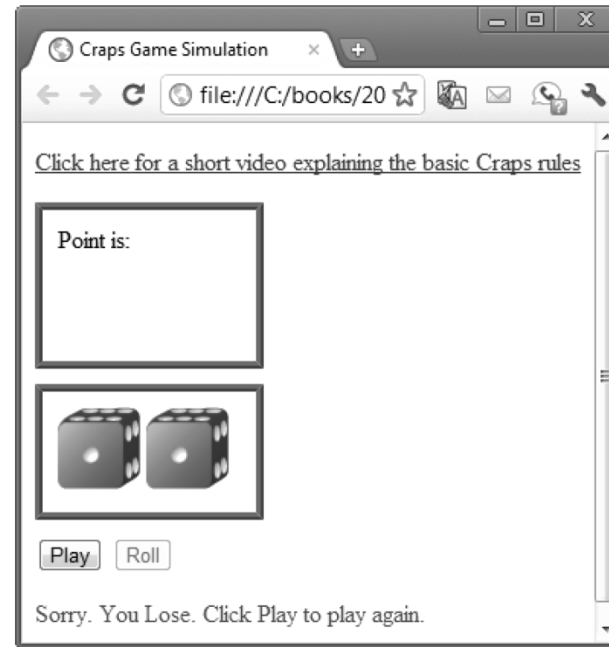
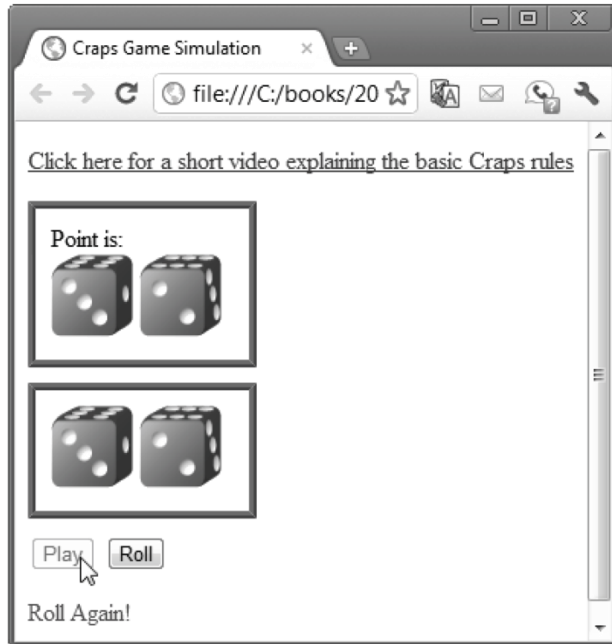


Fig. 9.7 | Craps game simulation. (Part 10 of 12.)

c) First roll is a 5, so the user's point is 5. The **Play** button is disabled and the **Roll** button is enabled.

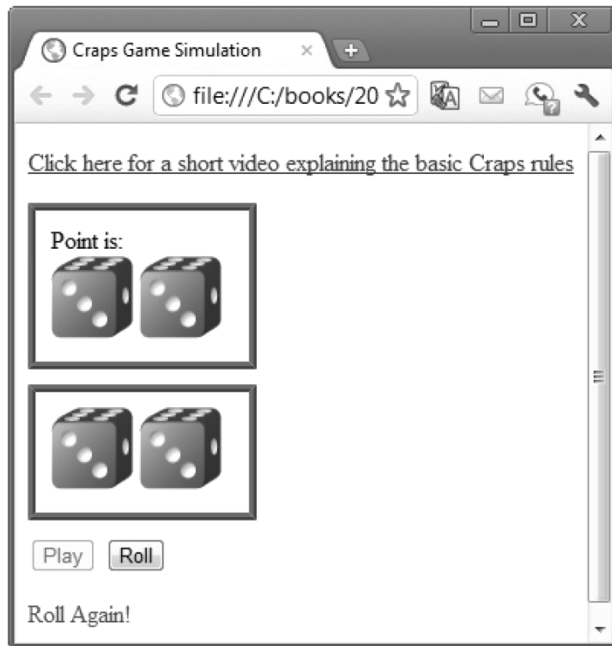


d) User won on a subsequent roll. The **Play** button is enabled and the **Roll** button is disabled.



Fig. 9.7 | Craps game simulation. (Part 11 of 12.)

e) First roll is a 6, so the user's point is 6. The **Play** button is disabled and the **Roll** button is enabled.



f) User lost on a subsequent roll. The **Play** button is enabled and the **Roll** button is disabled.

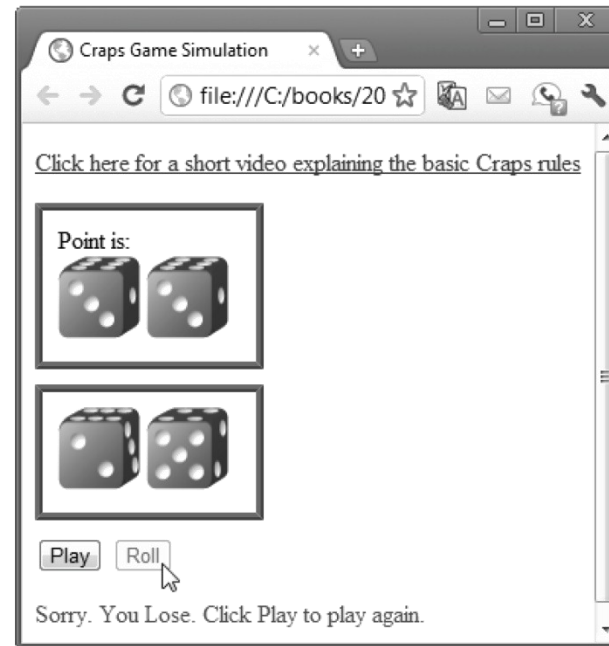


Fig. 9.7 | Craps game simulation. (Part 12 of 12.)



9.6 Example: Game of Chance; Introducing the HTML audio and video Elements (Cont.)

The HTML audio Element

- ▶ An HTML audio element is used to embed audio into a web page.
- ▶ We specify an `id` for the element, so that we can programmatically control when the audio clip plays, based on the user's interactions with the game.
- ▶ Setting the `preload` attribute to "**auto**" indicates to the browser that **it should consider downloading the audio clip** so that it's ready to be played when the game needs it.
 - 多數瀏覽器限制自動播放與預載行為，且行動裝置對流量敏感。
 - 音訊播放通常需要使用者互動才允許



9.6 Example: Game of Chance; Introducing the HTML audio and video Elements (Cont.)

- ▶ Most browsers support MP3, OGG and/or WAV format.
- ▶ Each source element specifies a `src` and a `type` attribute.
 - The `src` attribute specifies the location of the audio clip.
 - The `type` attribute specifies the clip's MIME type—`audio/mpeg` for the MP3 clip (WAV would be `audio/wav`; MIME types for these and other formats can be found online).
- ▶ When a web browser that supports the audio element encounters the source elements, it will choose the first audio source that represents one of the browser's supported formats.



9.6 Example: Game of Chance; Introducing the HTML audio and video Elements (Cont.)

CrapsRules.html and the HTML5 video Element

- ▶ When the user clicks the hyperlink in Craps.html, the CrapsRules.html is displayed in the browser.
- ▶ This page consists of a link back to Craps.html (Fig. 9.8) and an HTML video element that displays a video explaining the basic rules for the game of Craps.


```
1  <!DOCTYPE html>
2
3  <!-- Fig. 9.8: CrapsRules.html -->
4  <!-- Web page with a video of the basic rules for the dice game Craps. -->
5  <html>
6      <head>
7          <meta charset = "utf-8">
8          <title>Craps Rules</title>
9      </head>
10     <body>
11         <p><a href = "Craps.html">Back to Craps Game</a></p>
12         <video controls>
13             <source src = "CrapsRules.mp4" type = "video/mp4">
14             <source src = "CrapsRules.webm" type = "video/webm">
15             A player rolls two dice. Each die has six faces that contain
16             one, two, three, four, five and six spots, respectively. The
```

Fig. 9.8 | Web page that displays a video of the basic rules for the dice game Craps. (Part 1 of 3.)



```
17         sum of the spots on the two upward faces is calculated. If the
18         sum is 7 or 11 on the first throw, the player wins. If the sum
19         is 2, 3 or 12 on the first throw (called "craps"), the player
20         loses (i.e., the "house" wins). If the sum is 4, 5, 6, 8, 9 or
21         10 on the first throw, that sum becomes the player's "point."
22         To win, you must continue rolling the dice until you "make your
23         point" (i.e., roll your point value). You lose by rolling a 7
24         before making the point.
25     </video>
26 </body>
27 </html>
```

Fig. 9.8 | Web page that displays a video of the basic rules for the dice game Craps. (Part 2 of 3.)

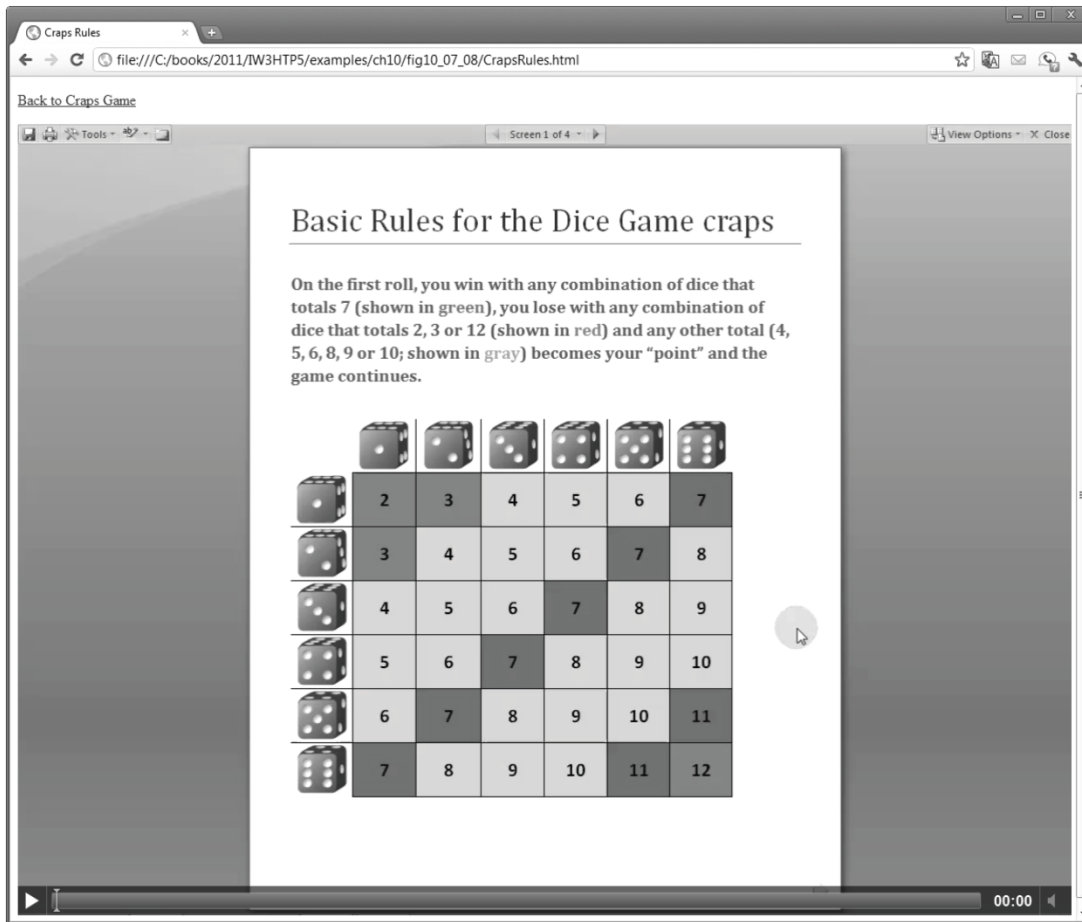


Fig. 9.8 | Web page that displays a video of the basic rules for the dice game Craps. (Part 3 of 3.)

Function Hoisting

- ▶ Hoisting is JavaScript's default behavior of moving declarations to the top of the current scope.
 - Hoisting applies to variable declarations and to function declarations.

```
myFunction(5);  
  
function myFunction(y) {  
    return y * y;  
}
```

<https://www.w3schools.com/code/tryit.asp?filename=FS09SRRZL454>



Immediately-Invoking Function Expressions (IIFE)

- ▶ Function expressions can be made "self-invoking".
 - Function expressions will execute automatically if the expression is followed by ().

```
(function () {  
    let x = "Hello!!";           // I will invoke myself  
})();
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_function_expression_self



JavaScript Nested Functions

- ▶ JavaScript supports nested functions.
 - Nested functions have **access to the scope "above" them**.

```
function add() {  
    let counter = 0;  
    function plus() {counter += 1;}  
    plus();  
    return counter;  
}
```

JavaScript Closures₁

- ▶ A closure is a function having access to the parent scope, even after the parent function has closed.

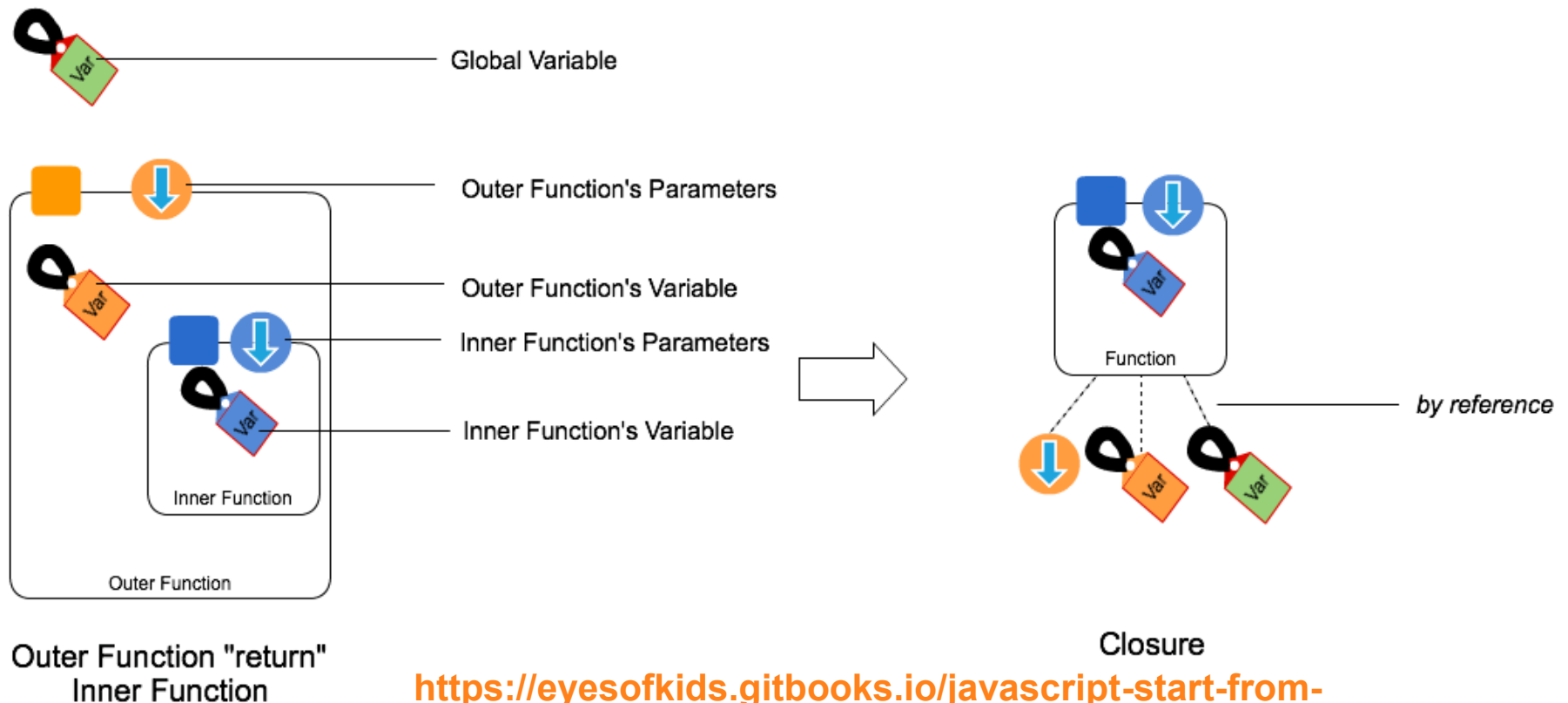
```
const add = (function () {  
    let counter = 0;  
    return function () {counter += 1; return counter;}  
})();
```

```
add();  
add();  
add();
```

The counter is protected by the scope of the anonymous function, and can only be changed using the add function.

https://www.w3schools.com/js/tryit.asp?filename=tryjs_function_closures5

JavaScript Closures₂



<https://eyesofkids.gitbooks.io/javascript-start-from-es6/content/part4/closure.html>

[js-07-closure.html](#)

[js-08-closure-2.html](#)



ES6: Default Parameter Values

- ▶ ES6 allows function parameters to have default values.

```
function myFunction(x, y = 10) {  
    // y is 10 if not passed or undefined  
    return x + y;  
}  
myFunction(5); // will return 15
```

https://www.w3schools.com/js/tryit.asp?filename=tryjs_es6_default